📘 Guía de Ejercicios Programación

📚 Información General

🎓 **Universidad:** Universidad Nacional de Lanús
💻 **Materia:** Programación y Programación de Computadoras
👩‍🏫 **Docente:** Esp. Lic. Nicolás Ignacio Pérez
📧 **E-mail de contacto:** niperez@unla.edu.ar
📂 **Repositorio oficial:** link

✨ Índice

🔹 **BLOQUE I: Introducción a C** **4**
🔹 **BLOQUE II: Arrays y Números Aleatorios** **6**
🔹 **BLOQUE III: Arrays, Bidimensionales, Ordenamiento y Búsqueda** **7**
🔹 **BLOQUE IV: Desafíos Generales (Figuras con bucles)** 🎨✨ **8**
📘 **Bloque IV: Estructuras en C** **9**
🧍 Ejercicio 1: Persona simple 9
🚚 Ejercicio 2: Vehículo simple 9
⚽ Ejercicio 3: Jugador simple 9
🏟️ Ejercicio 4: Estadio simple 9
🔄 Ejercicio 5: Comparar personas 10
🔗 **Bloque IV-bis:Estructuras anidadas** **11**
🚐 Ejercicio 6: Vehículo con pasajeros 11
🎮 Ejercicio 7: Equipo de fútbol 11
🚌 Ejercicio 8: Ordenar pasajeros de un vehículo 11
🏟️ Ejercicio 9: Estadio con equipos 11
📊 Ejercicio 10: Estadísticas de un vehículo 12
📘 **Bloque V: Punteros en C** **13**
🔢 Ejercicio 1: Puntero a entero básico 13
🔄 Ejercicio 2: Intercambio con punteros 13
➕ Ejercicio 3: Sumar con punteros 13
📚 Ejercicio 4: Arreglo y punteros 13
🔍 Ejercicio 5: Buscar en arreglo con punteros 14
🧍 **Punteros y estructuras** **15**
🧍 Ejercicio 6: Puntero a persona 15
🚚 Ejercicio 7: Puntero a vehículo 15
⚽ Ejercicio 8: Array de estructuras con punteros 15
🏟️ Ejercicio 9: Función que recibe puntero a estructura 15
🔄 Ejercicio 10: Ordenar estructuras con punteros 16
📘 **Bloque VI: Modularización con .h** **17**
🧍 Ejercicio 1: Persona con typedef y punteros 17
🚚 Ejercicio 2: Vehículo modular 17
⚽ Ejercicio 3: Jugador con array dinámico 17
🏟️ Ejercicio 4: Estadio con equipos 18
🧮 Ejercicio 5: Vehículo con pasajeros dinámico 18
💾 **Bloque V: Archivos y Persistencia** **19**
📚 Objetivo del bloque 19
🧍 Ejercicio 1: Guardar y leer personas en archivo de texto 19
🚚 Ejercicio 2: Guardar vehículos en archivo binario 19
🚌 Ejercicio 3: Estructura anidada — Vehículo con pasajeros 20
🔗 **Bloque VI: Listas Simplemente Enlazadas (int)** **22**
📚 Objetivo del bloque 22

🧩 Ejercicio 1: Insertar y mostrar lista..... 22
🎲 Ejercicio 2: Insertar al final y buscar en la lista..... 22
📉 Ejercicio 3: Eliminar nodo de la lista.....23
🔗 **Bloque VII: Listas Simplemente Enlazadas Genéricas (void*)..... 24**
📚 Objetivo del bloque.....24
🧩 Ejercicio 1: Lista genérica con enteros.....24
🎲 Ejercicio 2: Lista genérica con float.....24
⚡ Ejercicio 3: Callback para mostrar cualquier tipo de dato..... 25
🎥 **Bloque Extra: Recursos en Video.....26**
📚 Objetivo del bloque.....26
🔗 Lista de Reproducción.....26

🔹 BLOQUE I: Introducción a C

1️⃣ Suma de dos números ➕
Pedir dos números por teclado, sumarlos y mostrar el resultado.

2️⃣ División de dos números ➗
Pedir dos números por teclado, dividirlos y mostrar el resultado.

3️⃣ Datos personales 🧍
Ingresar nombre, apellido, edad, altura y género de una persona. Mostrar los datos en pantalla.

4️⃣ Explicación de expresiones 📑
Analizar qué imprime cada `printf`:

```
int a = 1, b = 2;
printf("%d\n", 3==5);
printf("%d\n", 3<=5);
printf("%d\n", -3<=-5);
printf("%d\n", (3<=5)&&(3==3));
printf("%d\n", (a<=b)||(b==4));
printf("%d\n", (8 % 3));
```

5️⃣ Expresión lógica 🧮
Pedir `a` y `b` al usuario y mostrar el resultado de `(a==b)&&((3<b)||(b<a))`. Explicar el resultado.

6️⃣ Número par y múltiplo de 5 🔢
Ingresar un número y decir si es par y múltiplo de 5.

7️⃣ Mayor de dos números 🔝
Pedir dos números y mostrar el mayor.

8️⃣ Ordenar tres números 📊
Ingresar tres números y mostrarlos de mayor a menor.

9️⃣ Índice de Masa Corporal (IMC) ⚖️
Ingresar peso y altura, calcular el IMC.

🔟 Año bisiesto 📅
Ingresar un año y determinar si es bisiesto.

1️⃣1️⃣ Día de la semana 📅
Ingresar un número (1–7) y mostrar el día correspondiente. Otro valor: mostrar “ERROR”.

1️⃣ 2️⃣ **Números pares entre 15 y 50** 🎯
Mostrar todos los pares en ese rango.

1️⃣ 3️⃣ **Divisibles con condiciones** ✅
Mostrar todos los números menores a 10000 divisibles por 2, 6 y 7, que no sean divisibles por 4.

1️⃣ 4️⃣ **Número primo** 🔍
Ingresar un número y mostrar si es primo o no.

1️⃣ 5️⃣ **Factorial de un número** ✖️
Ingresar un número y calcular su factorial.

1️⃣ 6️⃣ **Prueba de escritorio** 📓
Analizar paso a paso los ejercicios 4 y 5 con `n = 6`.

1️⃣ 7️⃣ **Verificación de número primo** 🔎
Pedir un número y determinar si es primo.

1️⃣ 8️⃣ **Primeros 20 pares (while)** ⏳
Mostrar los primeros 20 números pares usando un `while`.

1️⃣ 9️⃣ **Número a binario** 💻
Pedir un número y mostrar su equivalente en binario.

2️⃣ 0️⃣ **Suma acumulada** ➕
Pedir números positivos y sumarlos hasta ingresar un número ≤ 0 o superar 1500.

2️⃣ 1️⃣ **Sucesión de Fibonacci** 🌀
Pedir `n` y mostrar el número en la posición `n` de Fibonacci.

2️⃣ 2️⃣ **Valor medio** 📐
Calcular la media de números ingresados por el usuario. Termina con `0`.

2️⃣ 3️⃣ **Operaciones mixtas** 🔄
Pedir 20 números enteros y calcular:

- La suma de los pares
- El producto de los impares
- La media de los mayores a 5
- El promedio de los mayores a 20

🔹 BLOQUE II: Arrays y Números Aleatorios

📌 Ahora trabajaremos **arrays de enteros**, generando números aleatorios con `time.h`.
📌 Se fomenta el uso de **funciones y procedimientos**.

Ejemplos:

1️⃣ Generar un array de n enteros aleatorios (1–100) y mostrarlo. 🎲
2️⃣ Cargar un array con enteros ingresados por el usuario y mostrar la suma. ➕
3️⃣ Generar un array aleatorio y mostrar el mayor y menor. 🔝 🔽
4️⃣ Calcular el promedio de los elementos de un array. 📐
5️⃣ Contar cuántos números pares e impares hay en un array. ⚖️
6️⃣ Contar cuántos múltiplos de 3 hay en un array aleatorio. 🎯
7️⃣ Copiar los elementos de un array en otro. 📋
8️⃣ Invertir el contenido de un array. 🔄
9️⃣ Generar un array y mostrar cuántos valores superan el promedio. 📊
🔟 Sumar dos arrays elemento a elemento. ➕ ➕
1️⃣1️⃣ Multiplicar dos arrays elemento a elemento. ✖️
1️⃣2️⃣ Generar dos arrays y crear un tercero con la diferencia absoluta. ➖
1️⃣3️⃣ Crear una función que devuelva el valor máximo del array. 🔎
1️⃣4️⃣ Crear una función que devuelva la posición del mínimo. 📍
1️⃣5️⃣ Buscar un número dentro de un array y decir si existe. 🔍

🔹 BLOQUE III: Arrays, Bidimensionales, Ordenamiento y Búsqueda

📌 Ahora usamos **arrays `int` y `float`**, unidimensionales y bidimensionales.
📌 Implementamos **ordenamientos (inserción, selección, burbuja)** y **búsquedas (lineal, binaria)**.

1️⃣ Ordenar un array de enteros aleatorios con **burbuja**. 🫧
2️⃣ Ordenar un array de enteros aleatorios con **selección**. 🔽
3️⃣ Ordenar un array de enteros aleatorios con **inserción**. 📌
4️⃣ Buscar un valor en un array con búsqueda **lineal**. 🔍
5️⃣ Buscar un valor en un array ordenado con **binaria**. 🎯
6️⃣ Generar una matriz 3x3 con aleatorios y mostrarla. 🔢
7️⃣ Sumar todos los elementos de una matriz NxM. ➕
8️⃣ Mostrar la diagonal principal de una matriz cuadrada. 📐
9️⃣ Calcular la suma de cada fila y columna de una matriz. 📊
🔟 Transponer una matriz cuadrada. ↔️
1️⃣1️⃣ Generar un array de floats y calcular la media. 🧮
1️⃣2️⃣ Ordenar un array de floats con **burbuja**. 🫧
1️⃣3️⃣ Comparar la eficiencia de burbuja, selección e inserción con arrays grandes. ⏱️
1️⃣4️⃣ Buscar el máximo en una matriz NxM. 🔝
1️⃣5️⃣ Ordenar todos los elementos de una matriz en un único array. 📦

🔹 BLOQUE IV: Desafíos Generales (Figuras con bucles) 🎨✨

1️⃣ **Pirámide de asteriscos** 🏔️
Dibujar una pirámide de altura `n`.

2️⃣ **Triángulo invertido** 🔻
Dibujar un triángulo de asteriscos invertido, con base de tamaño `n`.

3️⃣ **Rombo** 💎
Dibujar un rombo de altura `n`.

4️⃣ **Tablero de ajedrez** ♟️
Dibujar un tablero NxN usando `*` y `-`.

5️⃣ **Escalera de números** 📶
Dibujar una escalera donde cada fila contiene números consecutivos.

⚠️ Todos estos ejercicios deben resolverse con **funciones parametrizadas** (`altura`, `ancho`, `carácter a usar`), para que las figuras puedan modificarse fácilmente.

📘 Bloque IV: Estructuras en C

🧍 Ejercicio 1: Persona simple

👉 Definir una estructura `Persona` con nombre, edad y altura.

- Pedir los datos por teclado.
- Mostrar la persona con una función `mostrarPersona`.

🚗 Ejercicio 2: Vehículo simple

👉 Definir una estructura `Vehiculo` con **marca**, **modelo** y **año**.

- Pedir los datos de un vehículo.
- Mostrar toda la información por pantalla.

⚽ Ejercicio 3: Jugador simple

👉 Crear una estructura `Jugador` con **nombre**, **posición** y **goles convertidos**.

- Pedir los datos de un jugador.
- Mostrar su información usando una función.

🏟️ Ejercicio 4: Estadio simple

👉 Definir una estructura `Estadio` con **nombre**, **ciudad** y **capacidad**.

- Pedir los datos de un estadio.
- Mostrar toda la información.

🔄 Ejercicio 5: Comparar personas

👉 Definir **dos personas** y cargarlas por teclado.

- Mostrar quién es mayor en edad.
- Usar funciones para comparar.

🔗 Bloque IV-bis:Estructuras anidadas

🚐 Ejercicio 6: Vehículo con pasajeros

👉 Un `Vehiculo` puede tener hasta **5 pasajeros** de tipo `Persona`.

- Definir `Vehiculo` con un array de `Persona`.
- Cargar los datos del vehículo y de cada pasajero.
- Mostrar toda la información completa.

🎮 Ejercicio 7: Equipo de fútbol

👉 Un `Equipo` puede tener hasta **11 jugadores** de tipo `Jugador`.

- Definir la estructura `Equipo` con un array de `Jugador`.
- Cargar un equipo y mostrarlo en pantalla.

🚌 Ejercicio 8: Ordenar pasajeros de un vehículo

👉 Usando el `Vehiculo` con pasajeros:

- Ordenar a los pasajeros por **edad de menor a mayor**.
- Mostrar la lista ordenada.
- Implementar la ordenación en una función.

🏟️ Ejercicio 9: Estadio con equipos

👉 Un `Estadio` recibe hasta **2 equipos de fútbol**.

- Definir `Estadio` con **nombre**, **capacidad** y 2 equipos.
- Cargar todos los datos y mostrarlos.

📊 Ejercicio 10: Estadísticas de un vehículo

👉 Usando un `Vehiculo` con pasajeros:

- Calcular el **promedio de edad** de los pasajeros.
- Mostrar qué pasajero es el **más alto**.
- Implementar con funciones.

📘 Bloque V: Punteros en C

🔢 Ejercicio 1: Puntero a entero básico

👉 Declarar un entero y un puntero a entero.

- Mostrar la dirección de memoria y el valor con `*ptr`.
- Cambiar el valor usando el puntero.

🔄 Ejercicio 2: Intercambio con punteros

👉 Crear una función `intercambiar(int *a, int *b)` que intercambie dos valores enteros usando punteros.

- Probar con dos números cargados por teclado.

➕ Ejercicio 3: Sumar con punteros

👉 Crear una función `sumar(int *a, int *b, int *resultado)` que calcule la suma de dos enteros usando punteros.

- Mostrar el resultado desde el `main`.

📚 Ejercicio 4: Arreglo y punteros

👉 Declarar un arreglo de enteros y recorrerlo usando **aritmética de punteros** (`*(ptr+i)`).

- Mostrar todos los valores.

🔍 Ejercicio 5: Buscar en arreglo con punteros

👉 Implementar una función que reciba un arreglo y su tamaño como puntero.

- Buscar un valor dentro del arreglo.
- Devolver la posición encontrada o `-1`.

🧍 Punteros y estructuras

🧍 Ejercicio 6: Puntero a persona

👉 Definir `struct Persona { char nombre[20]; int edad; };`

- Declarar un puntero a `Persona`.
- Cargar los datos de una persona desde el puntero.
- Mostrar usando `->`.

🚗 Ejercicio 7: Puntero a vehículo

👉 Definir `struct Vehiculo { char marca[20]; int modelo; };`

- Crear una función que reciba un puntero a `Vehiculo` y lo cargue.
- Mostrar los datos en el `main`.

⚽ Ejercicio 8: Array de estructuras con punteros

👉 Definir un array de `Jugador`.

- Usar punteros para recorrer el array y cargar los jugadores.
- Mostrar la lista de jugadores con un puntero.

🏟️ Ejercicio 9: Función que recibe puntero a estructura

👉 Definir un `Estadio`.

- Crear una función que reciba un puntero a `Estadio` y modifique su capacidad.
- Mostrar antes y después del cambio.

🔄 Ejercicio 10: Ordenar estructuras con punteros

👉 Definir un array de `Persona`.

- Usar punteros para ordenar las personas por edad (burbuja con punteros).
- Mostrar la lista ordenada.

📘 Bloque VI: Modularización con `.h`

🧍 Ejercicio 1: Persona con typedef y punteros

👉 Objetivo: separar en archivos `.h` y `.c`.

- Crear `persona.h` con la declaración de la estructura `Persona` usando `typedef` y prototipos de funciones:
  - `Persona* crearPersona()`,
  - `void mostrarPersona(Persona* p)`.
- Implementar `persona.c` con las funciones.
- `main.c` debe crear un `Persona*`, cargar datos y mostrarlos.

🚗 Ejercicio 2: Vehículo modular

👉 Definir un `typedef struct Vehiculo` en `vehiculo.h`.

- Funciones:
  - `Vehiculo* crearVehiculo()`
  - `void mostrarVehiculo(Vehiculo* v)`
- Implementar en `vehiculo.c` y probar en `main.c`.
- Usar punteros para manejar la estructura.

⚽ Ejercicio 3: Jugador con array dinámico

👉 Definir `Jugador` con `typedef` y punteros.

- Funciones en `jugador.h`:
  - `Jugador* crearJugador()`
  - `void mostrarJugador(Jugador* j)`
- En `main.c` crear un **array dinámico de jugadores** usando punteros (`Jugador* jugadores = malloc(n * sizeof(Jugador))`) y mostrarlo.

🏟️ Ejercicio 4: Estadio con equipos

👉 Definir `typedef struct Estadio` que tenga **2 punteros a `Equipo`**.

- Funciones:
  - `Estadio* crearEstadio()`
  - `void mostrarEstadio(Estadio* e)`
- Implementar en `.c` y usar en `main.c`.
- Los punteros a `Equipo` permiten tener dinámicamente hasta 11 jugadores por equipo.

🧮 Ejercicio 5: Vehículo con pasajeros dinámico

👉 `Vehiculo` tiene un **array dinámico de `Persona*`** (máximo 5).

- Funciones:
  - `Vehiculo* crearVehiculoConPasajeros(int nPasajeros)`
  - `void mostrarVehiculoConPasajeros(Vehiculo* v)`
- Implementar `.h` y `.c`.
- En `main.c`: crear vehículo, asignar pasajeros y mostrar información usando punteros.

💾 Bloque V: Archivos y Persistencia

📚 Objetivo del bloque

Aprender a **guardar y leer datos de estructuras** en archivos de texto y binarios.
Fomentar el uso de **`typedef`** + **`struct`**, y la separación en **`.h`** y **`.c`** para organizar el código.

🧍 Ejercicio 1: Guardar y leer personas en archivo de texto

👉 Crear un programa con la estructura:

```
typedef struct Persona{
    char nombre[20];
    int edad;
} PersonaPtr;
```

- Definir `Persona` en un `.h`.
- En el `.c`, hacer funciones para:
  - Cargar un arreglo de personas.
  - Guardar las personas en un archivo de texto (`personas.txt`).
  - Leer el archivo y mostrar las personas.

🚚 Ejercicio 2: Guardar vehículos en archivo binario

👉 Crear un programa con la estructura:

```
typedef struct Vehiculo{
```

```
    char marca[20];
    int modelo;
} VehiculoPtr;
```

- Definir `Vehiculo` en un `.h`.
- En el `.c`, implementar funciones que:
  - Carguen vehículos en un array.
  - Guarden todo el array en un archivo binario (`vehiculos.txt`).
  - Lean el archivo binario y muestren los vehículos.

🚌 Ejercicio 3: Estructura anidada — Vehículo con pasajeros

👉 Crear un programa con la estructura:

```
typedef struct Persona {
    char nombre[20];
    int edad;
} PersonaPtr;

typedef struct Vehiculo {
    char marca[20];
    int modelo;
    PersonaPtr pasajeros[5];  // hasta 5 pasajeros
} VehiculoPtr;
```

- Definir ambas estructuras en un `.h`.
- En el `.c`, implementar funciones para:
    - Cargar un vehículo con hasta 5 pasajeros.
    - Guardar el vehículo completo en un archivo binario (`vehiculo_con_pasajeros.txt`).
    - Leerlo desde archivo y mostrar los datos del vehículo y sus pasajeros.

🔗 Bloque VI: Listas Simplemente Enlazadas (int)

📚 Objetivo del bloque

Comprender cómo manejar **estructuras dinámicas** en C a través de **listas enlazadas**. Practicar la definición de nodos con `typedef struct`, el uso de punteros y la implementación de funciones de inserción, recorrido y búsqueda.

🧩 Ejercicio 1: Insertar y mostrar lista

👉 Definir la estructura del nodo en un `.h`:

```
typedef struct Nodo {

    int dato;

    struct Nodo* siguiente;

} Nodo;
```

- Crear funciones en el `.c` para:
  - Insertar un nuevo nodo al inicio de la lista.
  - Recorrer y mostrar toda la lista.
- Probar en el `main.c` insertando al menos 5 números aleatorios en la lista.

🎲 Ejercicio 2: Insertar al final y buscar en la lista

👉 Usando la misma estructura `Nodo`:

- Hacer funciones en el `.c` para:

    - Insertar un nodo al final de la lista.
    - Buscar un valor en la lista (retornar puntero al nodo o `NULL` si no existe).
- Probar en el `main.c` con valores aleatorios y búsquedas.

📉 Ejercicio 3: Eliminar nodo de la lista

👉 Usando la misma estructura `Nodo`:

- Implementar funciones en el `.c` para:
    - Eliminar el primer nodo que tenga un valor específico.
    - Mostrar la lista luego de la eliminación.
- En el `main.c`, generar una lista con números aleatorios, mostrarla, pedir un valor al usuario y eliminarlo si existe.

🔗 Bloque VII: Listas Simplemente Enlazadas Genéricas (void*)

📚 Objetivo del bloque

Aprender a:

- Definir listas genéricas con punteros a `void`.
- Manejar inserción, recorrido y eliminación sin depender del tipo de dato.
- Usar **funciones callback** para trabajar con los datos almacenados.

🧩 Ejercicio 1: Lista genérica con enteros

👉 Definir la estructura en un `.h`:

```
typedef struct Nodo {
    void* dato;
    struct Nodo* siguiente;
} NodoPtr;
```

- Implementar funciones en un `.c` para:
  - Insertar un nuevo nodo al inicio.
  - Recorrer la lista mostrando enteros (`int*`).
- Probar en `main.c` creando una lista de enteros.

🎲 Ejercicio 2: Lista genérica con `float`

👉 Usando la misma estructura:

- Implementar funciones en un `.c` para:
  - Insertar al final de la lista.
  - Recorrer mostrando valores `float*`.
- Probar en `main.c` con números decimales generados aleatoriamente.

⚡ Ejercicio 3: Callback para mostrar cualquier tipo de dato

👉 Usando la misma estructura:

- Implementar en el `.c`:
  - Una función `recorrerLista(Nodo* lista, void (*mostrar)(void*))` que reciba un callback para mostrar los elementos.
- Crear dos callbacks en el `main.c`:
  - `mostrarEntero(void* dato)`
  - `mostrarFloat(void* dato)`
- Probar generando dos listas: una de enteros y otra de floats, y recorrerlas con sus respectivos callbacks.

📽️ Bloque Extra: Recursos en Video

📚 Objetivo del bloque

Complementar la práctica con explicaciones visuales y ejemplos paso a paso a través de una **lista de reproducción de YouTube** enfocada en programación en C y fundamentos prácticos.

🔗 Lista de Reproducción

👉 Haz clic aquí para ver la lista completa